

CodeBlue

Real-Time Ambulance Dispatch & Emergency Response Platform

Hackathon Project Submission Document - Road Safety Hackathon 2026

Team: CtrlAltDuo

Members: Kanak Bansal, Swarnim Pandey

GitHub: <https://github.com/CtrlAltDuo/CodeBlue>

1. Problem Statement

Every year, thousands of preventable deaths occur in India because an ambulance simply doesn't arrive in time. The existing emergency infrastructure is incredibly fragmented — each hospital manages its own fleet with phone-based dispatch, there is zero real-time visibility for the person waiting, and there is no centralized coordination across hospitals. When a citizen dials for help, the response depends entirely on luck — which hospital picks up the call, whether they have a free vehicle, and whether they can find the address. There is no system that automatically picks the closest available ambulance from across the entire city, and there is certainly no way for the citizen to track its arrival in real time.

We built CodeBlue to fix this. Our goal was straightforward: build a platform where a person in an emergency can request help with a single tap, and the system automatically assigns the nearest available ambulance from any hospital in the network, then lets them track it live until it arrives at their doorstep.

2. What is CodeBlue?

CodeBlue is a full-stack, real-time ambulance dispatch and emergency response optimization platform. It connects four key stakeholders — citizens, ambulance drivers, hospital staff, and city-level administrators — through a unified web application. The platform handles everything from the moment a citizen requests an ambulance, through the intelligent automatic assignment of the closest available unit, all the way to live GPS tracking, ETA prediction, and post-incident analytics.

Think of it as Uber for ambulances, but purpose-built for the chaos of Indian emergency services, where every second counts.

3. Key Features

3.1 Citizen Emergency Interface

The citizen-facing interface is designed to be used under extreme stress. We intentionally kept it dead simple — no sign-up, no login, no unnecessary screens. A person in an emergency should never have to create an account first.

- **No login required.** Zero-friction access:
- The citizen just enters their phone number and drops a pin on the map to mark their pickup location. They can also use the GPS button to auto-detect their current location via the browser's Geolocation API.
- **When using GPS, the app automatically calls the Nominatim OpenStreetMap API to convert raw latitude/longitude into a human-readable address.** Reverse geocoding:
- **Before requesting, the citizen can see a list of nearby hospitals with live bed availability, ICU counts, available ambulances, and estimated ETA — all calculated using real-time Haversine distance from their current position.** Nearby hospital cards:

- Pressing "GET HELP NOW" creates an incident call on the backend, which instantly triggers the auto-dispatch engine. One-tap request:
- Once assigned, the citizen sees the ambulance's real-time position on the map, the driver's name, the vehicle plate number, the dispatching hospital name and address, and a live ETA countdown that recalculates dynamically as the ambulance moves. Live ambulance tracking:
- If the ambulance gets stuck or the citizen isn't satisfied, they can hit "Reassign" within 2 minutes. The system will free the current ambulance and dispatch the next closest one — a feature we haven't seen in any existing Indian emergency app. 2-minute reassignment window:

3.2 Admin Dispatch Center

This is the nerve center of the whole operation. The admin gets a God's-eye view of the entire city's emergency fleet.

- A dark-themed Leaflet map shows every ambulance (color-coded by status: green = available, yellow = en route, red = occupied) and every active emergency call (animated pulsing red markers). Global dispatch map:
- All location changes, new dispatch events, and call completions push to the admin map instantly via Socket.io — there is no page refreshing. Real-time updates via WebSockets:
- A collapsible sidebar shows real-time counts: total active calls, available ambulances, en-route ambulances, and occupied ambulances. Live fleet status panel:
- A toggle-able overlay that renders semi-transparent circles over completed incident locations, helping admins visually identify high-demand emergency zones. Incident cluster heatmap:
- The server aggregates historical call data and identifies the top-5 hotspot zones (by rounding GPS coordinates and counting calls). Toggling this overlay shows blue circles on the map, suggesting where idle ambulances should be strategically parked to minimize future response times. Predictive pre-positioning zones:

3.3 Analytics Dashboard

Beyond just dispatching, we give admins data-driven insights to continuously improve their fleet's performance.

- A line chart (built with Recharts) showing daily emergency call counts over the past month. 30-day call volume trend:
- A horizontal bar chart ranking the top-10 busiest districts by average response time in minutes. District-level response times:
- A detailed table showing each ambulance's license plate, current status, total completed calls, and average en-route time — making it easy to spot underperforming or overworked vehicles. Ambulance utilization table:
- Today's total calls, average response time, busiest district, and fleet availability percentage — all at a glance. Summary KPI cards:

3.4 Hospital Staff Dashboard

Each hospital that joins the CodeBlue network gets its own management dashboard.

- **A multi-step registration form where a hospital admin provides hospital details (name, address, GPS coordinates, phone, total beds, ICUs) and can register their ambulance fleet — each with a driver name, email, password, number plate, shift, and ID proof.**Hospital onboarding flow:
- The entire registration is wrapped in a database transaction — if anything fails mid-way, nothing is committed, keeping the data clean.
- **Hospital staff can view all their ambulances, see current driver assignments, and monitor vehicle statuses in real time.**Fleet management:
- **Staff can update the hospital's available beds and ICU count, which is then immediately visible to citizens searching for nearby hospitals.**Bed and ICU inventory:

3.5 Ambulance Driver Interface

Drivers get a streamlined, mobile-friendly interface that keeps things minimal so they can focus on driving.

- **When the system assigns a call to their ambulance, a real-time Socket.io push notification pops up with the pickup location details.**Incoming dispatch alerts:
- **Drivers can accept an incoming dispatch or reject it (which triggers admin notification).**Accept or reject calls:
- **Drivers can switch between Available, En Route, Occupied, and Offline — these changes are immediately reflected on the admin map and in the dispatch engine.**Status toggling:
- **The driver app pushes GPS updates back to the server, which relays them to the citizen's tracking view and the admin map simultaneously.**Live location broadcasting:

4. Technical Architecture

4.1 Architecture Overview

CodeBlue follows a monorepo architecture using npm workspaces. The repository is split into three packages — client, server, and shared. The shared package contains TypeScript type definitions (User, Hospital, Ambulance, IncidentCall, DispatchAssignment) that are imported by both client and server, keeping our data contracts in sync without any duplication.

The client and server communicate through two channels: a standard REST API for CRUD operations and authentication, and persistent WebSocket connections (via Socket.io) for all real-time events like location updates, dispatch alerts, and call status changes.

4.2 Tech Stack

Layer	Technology	Purpose
Frontend Framework	React 19 + TypeScript	Component-based UI with type safety
Build Tool	Vite 8	Near-instant HMR for rapid development
Styling	TailwindCSS v4	Utility-first CSS with responsive dark mode support
Maps	Leaflet + react-leaflet	Interactive geospatial visualization with custom markers
Charts	Recharts	Responsive data visualization for the analytics dashboard
Real-time (Client)	socket.io-client	WebSocket connections for live location and dispatch events
Backend Framework	Express.js 5 + TypeScript	RESTful API with typed request/response handling
Real-time (Server)	Socket.io	Bi-directional event-driven communication hub
Database	PostgreSQL (Supabase)	Cloud-hosted relational database with zero local setup
DB Driver	pg (node-postgres)	Raw SQL queries for maximum control and performance
Authentication	JWT + bcryptjs	Stateless token-based auth with hashed passwords
Security	express-rate-limit, CORS, cookie-parser	API rate limiting, cross-origin policy, secure cookies
Validation	express-validator + custom regex	Server-side input validation for emails, phones, plates
Monorepo	npm workspaces + concurrently	Shared types and simultaneous dev server startup

4.3 The Dispatch Engine — How It Works

The dispatch engine is the brain of CodeBlue. When a citizen hits "GET HELP NOW", here's exactly what happens under the hood:

- The client POSTs the citizen's phone number, pickup latitude, pickup longitude, and pickup address to `/api/calls`.
- The server creates a new `incident_call` row in the database with status 'pending'.
- The `autoDispatch()` function is called. It queries all ambulances with status = 'available' from the database.

- It calculates the Haversine distance (great-circle distance over the Earth's surface, using a radius of 6,371 km) between the citizen's pickup coordinates and each available ambulance's current GPS position.
- The ambulance with the shortest distance wins. The ETA is estimated using the formula: $ETA \text{ (minutes)} = (\text{distance_km} / 40) \times 60$, assuming an average urban ambulance speed of 40 km/h.
- A dispatch_assignment record is created linking the call to the ambulance. The ambulance status is updated to 'en_route' and the call status to 'assigned'.
- The server enriches the response with the driver's name, the vehicle's license plate, and the hospital's name and address by joining across the ambulances, users, and hospitals tables.
- A Socket.io event ('call_assigned') is emitted to both the admin room and the specific driver's room, ensuring instant notification.

If the citizen requests a reassignment within 2 minutes, the server validates the time window, excludes the current ambulance, re-runs the dispatch algorithm for the next closest unit, frees the old ambulance (setting it back to 'available'), and notifies the old driver to cancel via Socket.io.

5. Database Design

We use a relational PostgreSQL database hosted on Supabase. The schema is designed around six core tables, connected through foreign keys to maintain referential integrity.

Table	Key Columns	Purpose
hospitals	id, name, address, lat, lng, contact_phone, total_beds, available_beds, total_icus, available_icus	Stores each hospital's location, contact, and live capacity data.
users	id, name, email, password_hash, role (enum: admin / hospital_staff / driver / citizen), hospital_id (FK)	All users with role-based access control. Hospital staff and drivers are linked to their parent hospital.
driver_details	user_id (FK → users), shift, id_proof_number	Extended profile for driver users — stores shift type and government ID proof.
ambulances	id, hospital_id (FK), driver_id (FK), license_plate, status (enum), current_lat, current_lng	Each ambulance is linked to a hospital and driver. Stores real-time GPS and status.
incident_calls	id, citizen_id (FK), caller_phone, pickup_lat, pickup_lng, pickup_address, status (enum)	Every emergency request — tracks the citizen's location, phone, and the lifecycle of the call.
dispatch_assignments	id, call_id (FK), ambulance_id (FK), pickup_eta_minutes, assigned_at, completed_at	The junction that links a call to its assigned ambulance, with ETA and timestamps.

location_history	id, ambulance_id (FK), lat, lng, recorded_at	Chronological GPS breadcrumb trail for every ambulance. Used for post-incident analysis and route replay.
------------------	--	---

6. Real-Time Communication Architecture

Real-time communication is the backbone of CodeBlue. We use Socket.io with a room-based architecture:

- **All admin users subscribe to this room. They receive every location_update, call_assigned, call_rejected, and call_completed event across the entire system.**'admin' room:
- **When a citizen gets an ambulance assigned, they subscribe to this room. They receive only the location updates relevant to their specific call.**'call:{callId}' room:
- **Each driver subscribes to their ambulance's room. They receive dispatch alerts and cancellation events for their vehicle.**'driver:{ambulanceId}' room:
- **Hospital staff subscribe here to see location updates for ambulances belonging to their hospital.**'hospital:{hospitalId}' room:

Socket connections are authenticated using JWT tokens passed through the handshake. The middleware verifies the token before allowing the connection, preventing unauthorized access to the real-time event stream.

7. Security & Authentication

- **All passwords are hashed with bcryptjs using a salt round of 10 before being stored in the database. We never store plaintext passwords.**Password hashing:
- **On login, a signed JWT containing the user's id, email, role, and hospital_id is generated with a 7-day expiry. The token is stored in an httpOnly cookie (preventing XSS access) and also returned in the response for API usage.**JWT authentication:
- **The client uses a ProtectedRoute component that checks for a valid token and the user's role before rendering admin, hospital, or driver pages. Unauthorized users are redirected.**Route protection:
- **Custom regex validators on the server check email format, password strength (min 8 chars, uppercase, lowercase, digit), phone numbers (10-15 digits), and vehicle number plates before processing.**Server-side validation:
- **The server restricts cross-origin requests to the configured client URL only, with credentials enabled.**CORS policy:
- **express-rate-limit is integrated to prevent abuse of public-facing endpoints.**Rate limiting:

8. API Endpoints

Method & Path	Auth?	Description
POST /api/auth/register	No	Register a new user (admin, driver, citizen).
POST /api/auth/register-hospital	No	Multi-step hospital onboarding (hospital + staff + fleet) in a single transaction.
POST /api/auth/login	No	Authenticate and receive JWT token.
POST /api/auth/logout	No	Clear the auth cookie.
GET /api/auth/me	Yes	Fetch the currently authenticated user's profile.
POST /api/calls	No	Create a new emergency call and trigger auto-dispatch.
GET /api/calls	No	List calls with optional status and date range filters.
POST /api/calls/:id/reassign	No	Reassign a call to the next closest ambulance (within 2-min window).
PATCH /api/calls/:id/status	No	Update call status (e.g., mark as completed).
GET /api/ambulances	No	List all ambulances, with optional hospital_id filter.
PATCH /api/ambulances/:id/status	No	Update an ambulance's status.
PATCH /api/ambulances/:id/location	No	Push a GPS update for an ambulance and broadcast via Socket.io.
GET /api/hospitals/nearby	No	Find nearest hospitals by lat/lng with live ETA and bed counts.
GET /api/hospitals/:id	No	Get a specific hospital's details.
PATCH /api/hospitals/:id/inventory	Yes	Update available beds and ICUs (hospital_staff only).
GET /api/analytics/response-times	No	District-level response time aggregates.
GET /api/analytics/call-volume	No	30-day daily call volume trend.
GET /api/analytics/ambulance-utilization	No	Per-ambulance utilization and performance stats.
GET /api/analytics/heatmap-data	No	Raw completed-incident coordinates for heatmap overlay.
GET /api/analytics/preposition-zones	No	Top-5 recommended ambulance pre-positioning hotspots.
GET /health	No	Health check endpoint — returns server uptime and database connectivity.

9. User Roles & Application Routes

Role	Route	Auth Required?	Key Capabilities
Citizen	/citizen	No	Request ambulance, track live, reassign, view nearby hospitals.
Admin	/admin-map	Yes (admin)	Global dispatch map, fleet status, incident heatmap, predictive zones.
Admin	/analytics	Yes (admin)	30-day call trends, response times, ambulance utilization metrics.
Hospital Staff	/hospital	Yes (hospital_staff)	Manage fleet, update bed/ICU inventory, view dispatch status.
Driver	/driver	Yes (driver)	Accept/reject calls, toggle availability, broadcast live GPS.

10. Project Structure

The monorepo is organized as follows:

```
CodeBlue/
├── client/                                # React + Vite frontend
│   └── src/
│       ├── api/                          # Axios API client configuration
│       ├── components/                   # Shared components (Layout, Skeleton)
│       ├── contexts/                     # React contexts (Auth, Theme, Toast)
│       ├── hooks/                        # Custom React hooks
│       ├── lib/                          # Utility libraries
│       └── pages/                         # All page components
│           ├── Landing.tsx               # Public landing page
│           ├── CitizenApp.tsx            # Emergency request & tracking
│           ├── AdminMap.tsx              # Dispatch map with overlays
│           ├── AnalyticsDashboard.tsx    # Performance analytics
│           ├── HospitalDashboard.tsx     # Hospital fleet management
│           ├── HospitalOnboarding.tsx    # Hospital registration
│           ├── DriverApp.tsx             # Driver dispatch interface
│           ├── Login.tsx                 # Authentication
│           └── Register.tsx              # User registration
├── server/                               # Express.js backend
│   └── src/
│       ├── db/                           # Database pool and schema
│       ├── middleware/                   # JWT authentication middleware
│       └── routes/                       # REST API route handlers
```

├──	auth.ts	# Register, login, logout, me
├──	calls.ts	# Emergency calls + auto-dispatch
├──	ambulances.ts	# Fleet status and GPS updates
├──	hospitals.ts	# Nearby search + inventory
├──	analytics.ts	# Response times, volume, utilization
├──	services/	# Business logic (dispatch engine)
├──	jobs/	# Background tasks (pre-positioning)
├──	socket/	# Socket.io initialization and events
├──	seed.ts	# Database seeding script
├──	shared/	# Shared TypeScript type definitions
├──	types.ts	# User, Hospital, Ambulance, Call, etc.

11. Setup & Running Instructions

Getting CodeBlue running locally is straightforward:

- Clone the repository: `git clone https://github.com/CtrlAltDuo/CodeBlue.git` && `cd CodeBlue`
- Install all dependencies: `npm install` (this uses npm workspaces to install client, server, and shared packages in one go).
- Database: The project uses a centralized cloud PostgreSQL database hosted on Supabase. You do not need to run a local PostgreSQL instance.
- Environment variables: Create `.env` files in both the `client/` and `server/` directories using the provided `.env.example` files.
- Start development servers: `npm run dev` — this launches both the Vite frontend (port 5173) and Express backend (port 5000) concurrently.
- Seed the database (optional): Run the seed script to populate sample hospitals, drivers, ambulances, and a test incident for demo purposes.
- Health check: Visit `http://localhost:5000/health` to confirm the backend is running and connected to the database.

12. What Makes CodeBlue Different

There are a few things we're genuinely proud of that set this apart from a typical hackathon project:

- **This isn't a mockup or a demo** — every user flow from citizen request to driver tracking to admin monitoring is fully functional with real data flowing through the system. It actually works end-to-end.
- **We didn't just build a form that sends a notification.** The dispatch engine automatically identifies and assigns the closest available ambulance using Haversine geospatial calculations, and handles edge cases like no available ambulances gracefully. Intelligent auto-dispatch.
- **We couldn't find any existing Indian emergency app that lets a citizen request a different ambulance if they feel the assigned one is taking too long.** Our 2-minute reassignment window is a genuine innovation. The reassignment feature.

- **The system doesn't just react — it proactively analyzes historical call data to recommend where idle ambulances should be parked to minimize future response times.**Predictive pre-positioning.
- **We didn't cut corners — bcrypt password hashing, JWT with httpOnly cookies, CORS restrictions, rate limiting, input validation with regex, and role-based access control are all implemented.**Production-grade security.
- **Shared types between client and server eliminate data contract drift, and npm workspaces keep everything manageable.**Clean monorepo architecture.

13. Future Scope

If we had more time, here's what we'd build next:

- Integration with real-world routing APIs (Google Maps Directions or Mappls Route API) for turn-by-turn navigation and more accurate ETAs that account for traffic.
- Push notifications for drivers via Firebase Cloud Messaging or a progressive web app (PWA) setup.
- SMS alerts to citizens using Twilio or a similar service, so they get updates even without keeping the browser open.
- Machine learning-powered dispatch that factors in traffic, road closures, and historical ambulance speeds — not just straight-line distance.
- Multi-city deployment with tenant isolation and city-specific admin dashboards.
- A dedicated React Native mobile app for drivers with background GPS tracking.

14. Conclusion

CodeBlue started as a hackathon idea, but we built it like a product we'd actually want to exist in the world. Emergency response in India is broken — fragmented hospital fleets, no real-time tracking, and citizens left guessing whether help is even on the way. We've built a working, end-to-end platform that tackles all of these problems with intelligent dispatch, live tracking, and actionable analytics. Every feature you see is real, functional, and tested.

We believe that technology like this shouldn't just exist in hackathon demos — it should be running in cities, saving real lives. CodeBlue is our contribution toward that future.

— Thank you